

Array-programming languages like NumPy generalise operations in scalars to apply transparently to vectors, matrices and other higher-dimensional arrays. Python does not have a default array data type, and processing data with Python *lists* and *for* loops is dramatically slower compared to corresponding operations in compiled languages like FORTRAN, C and C++. NumPy comes to the rescue, with its dynamically typed environment for *array* computation, similar to basic Matlab. You can create a simple array with the *array* function in NumPy:

```
1 In[1]: import numpy as np
2
3 In[2]: a = np.array([1,2,3,4,5])
4
5 In[2]: b = np.array([6,7,8,9,10])
6
7 In[3]: type(b) #check datatype
8
9 Out[3]: type numpy.ndarray #array
10
11 In[4]: a+b
12
13 Out[4]: array([7,9,11,13,15])
```

You can also convert a simple array to a matrix array using the *shape* attribute.

```
1 In[1]: import numpy as np
2
3 In[5]: c = np.array([1,4,5,7,2,6])
4
5 In[6]: c.shape = (2,3)
6
7 In[7]: c
8
9 Out[7]: array([[1,4,5],[7,2,6]]) // converted to a 2 column
matrix
```

Matrix operations

Now let us take a look at some simple matrix operations. The following matrix can be simply defined as:

$$M = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

```
1 # Defining a matrix and matrix multiplication
2
3 In[1]: import numpy as np
4
5 In[2]: x = np.array([[1,2,3],[4,5,6],[7,8,9]])
6
7 In[3]: y = np.array([[1,4,5],[2,6,5],[6,8,3]]) #another
```

```
matrix
8
9 In[4]: z = np.dot(x,y) #matrix multiplication using dot
attribute
10
11 In[5]: z
12
13 Out[5]: z = [[[23, 40, 24], [50, 94, 63], [77, 146, 102]]]
```

You can also create matrices in NumPy using the *matrix* class. However, it's preferable to use arrays, since most NumPy functions return arrays, and not matrices. Moreover, matrix objects have a maximum of Rank-2. To hold Rank-3 data, you need an array. Also, arrays are closer in semantics to tensor algebra, compared to matrix objects.

The following example shows how to transpose a matrix and define a diagonal matrix:

```
1 In[7]: import numpy as np
2
3 In[8]: x = np.array([[1,2,3],[4,5,6],[7,8,9]])
4
5 In[7]: xT = np.transpose(x) #take transpose of the matrix
6
7 In[8]: xT
8
9 Out[8]: xT = [[[1,4,7],[2,5,8],[3,6,9]]]
10
11
12 In[9]: n = diag(range(1,4)) #defining a diagonal matrix
13
14 In[10]: n
15
16 Out[10]: n = [[[1,0,0],[0,2,0],[0,0,3]]]
```

Linear algebra

You can also solve linear algebra problems using the *linalg* package contained in SciPy. Let us look at a few more examples of calculating matrix inverse and determinant:

```
1 # Matrix Inverse
2
3 In[1]: import numpy as np
4
5 In[2]: m = np.array([[1,3,3],[1,4,3],[1,3,4]])
6
7 In[3]: np.linalg.inv(m) #take inverse with linalg.inv
function
8
9 Out[3]: array([[ -7, -3, -3], [-1, 1, 0], [-1, 0, 1]])
10
11
12 #Calculating Determinant
```